

Študijný materiál — ZAP2 2025

Java Enterprise: JPA, JAXB, REST

Kompletný prehľad príkladov, teórie a riešení

Obsah: JPA entitné triedy · DB schéma · JPQL/SQL dotazy · JAXB serializácia · REST služby

1. JPA — Java Persistence API

JPA (Java Persistence API) je štandard pre ORM (Object-Relational Mapping) v Jave. Umožňuje ukladať Java objekty do relačnej databázy bez ručného písania SQL. Najznámejšie implementácie sú Hibernate a EclipseLink.

1.1 Základné anotácie

Anotácia	Význam	Príklad
@Entity	Trieda sa mapuje na tabuľku v DB	@Entity class Osoba
@Id	Primárny kľúč entity	@Id long id;
@GeneratedValue	PK sa generuje automaticky	@GeneratedValue Long id;
@Embeddable	Trieda nie je entita, ale vložiteľná do inej	@Embeddable class Adresa
@Embedded	Atribút je typu @Embeddable	@Embedded Adresa adresa;
@Inheritance	Stratégia mapovania dedičnosti	@Inheritance(strategy=SINGLE_TABLE)
@JoinColumn	Cudzí kľúč (FK) v tabuľke	@JoinColumn(name="ADR_ID")
@OneToOne	Asociácia 1:1	@OneToOne Adresa adresa;
@OneToMany	Asociácia 1:M	@OneToMany List<Auto> auta;
@ManyToOne	Asociácia M:1	@ManyToOne Osoba majitel;
@ManyToMany	Asociácia M:N	@ManyToMany List<Kurz> kurzy;
mappedBy	Inverzná strana obojsmernej asociácie	@ManyToMany(mappedBy="zam")

1.2 Stratégie dedičnosti

SINGLE_TABLE — jedna tabuľka pre celú hierarchiu

Všetky podtriedy sú v jednej tabuľke. Stĺpec DTYPE rozlišuje typ záznamu.

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
class Osoba {
    @Id long id;
    String meno;
}
@Entity
class FyzickaOsoba extends Osoba {
    String rc;
}
```

id	DTYPE	meno	rc	ico
1	FyzickaOsoba	Jano	800101	NULL
2	PravnickaOsoba	ESET	NULL	12345678

💡 DTYPE = diskriminátor, JPA ho pridáva automaticky. Nevýhoda: veľa NULL hodnôt.

JOINED — tabuľka pre každú triedu

Nadtrieda aj každá podtrieda má vlastnú tabuľku. Spojené cez primárny kľúč (id podtriedy = FK na nadtriedu).

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
class Osoba {
    @Id long id;
    String meno;
}
@Entity
class FyzickaOsoba extends Osoba {
    String rc; // len tu, v tabuľke FYZICKAOSOBA
}
```

💡 Výhoda: žiadne NULL hodnoty. Nevýhoda: pomalšie dotazy kvôli JOIN-om.

TABLE_PER_CLASS — tabuľka pre každú konkrétnu triedu

Každá konkrétna (non-abstract) trieda má vlastnú tabuľku so VŠETKÝMI atribútmi vrátane zdedených.

💡 Žiadna tabuľka pre abstraktnú nadtriedu. Problém pri polymorfných dotazoch (UNION).

Stratégia	Tabuľky	DTYPE?	NULL hodnoty	Rýchlosť
SINGLE_TABLE	1 (pre celú hierarchiu)	✓ Áno	⚠ Veľa	✓ Rýchle
JOINED	1 na triedu	✓ Áno	✓ Žiadne	⚠ Pomalšie (JOIN)
TABLE_PER_CLASS	1 na konkrétnu triedu	✗ Nie	✓ Žiadne	⚠ UNION

1.3 Asociácie — M:N a mappedBy

Pri M:N asociácii musí byť mappedBy na JEDNEJ strane (inverzná strana). Vlastník nemá mappedBy a riadi prepojovaciu tabuľku.

```
// Vlastník asociácie (bez mappedBy)
@Entity
class FyzickaOsoba extends Osoba {
    @ManyToMany
    List<PravnickaOsoba> zamestnavatel;
}

// Inverzná strana (s mappedBy)
@Entity
class PravnickaOsoba extends Osoba {
    @ManyToMany(mappedBy = "zamestnavatel")
    List<FyzickaOsoba> zamestnanec;
}
```

💡 Bez mappedBy by JPA vytvoril 2 prepojovacie tabuľky namiesto jednej!

1.4 @Embedded vs. @Entity — kedy čo použiť

	@Embeddable + @Embedded	Samostatná @Entity
Vlastná tabuľka?	✗ Nie — stĺpce sa rozpustia do tabuľky vlastníka	✓ Áno — má vlastnú tabuľku
Vlastné ID?	✗ Nie	✓ Áno (@Id)
Kedy použiť	Adresa, Peniaze — hodnotové objekty bez identity	Osoba, Auto — entity s vlastnou identitou

2. Úloha A — JPA triedy podľa UML diagramu (8 bodov)

ZADANIE

- Navrhните a napíšte deklarácie tried podľa UML diagramu.
- UML obsahuje: Osoba (id, meno) → nadtrieda
- FyzickaOsoba (rc, narod) → dedí z Osoba
- PravnickaOsoba (ico, dic) → dedí z Osoba
- Adresa (mesto, ulica) → vzťah 1:1 s Osoba
- FyzickaOsoba ↔ PravnickaOsoba: M:N (zamestnanec – zamestnávateľ)

Riešenie 1 — Adresa ako @Embeddable

Adresa sa 'rozpustí' do tabuľky OSOBA. Nevznikne samostatná tabuľka ADRESA.

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
class Osoba {
    @Id
    long id;
    String meno;
    @Embedded                // Adresa sa rozpustí do tabuľky OSOBA
    Adresa adresa;
}

@Embeddable                // Nemá vlastnú tabuľku!
class Adresa {
    String mesto;
    String ulica;
}

@Entity
class FyzickaOsoba extends Osoba {
    String rc;
    Date narod;
    @ManyToMany
    List<PravnickaOsoba> zamestnavatel;
}

@Entity
class PravnickaOsoba extends Osoba {
    String ico;
    String dic;
    @ManyToMany(mappedBy = "zamestnavatel")
    List<FyzickaOsoba> zamestnanec;
}
```

Riešenie 2 — Adresa ako samostatná entita, FK v Osobe

Adresa má vlastnú tabuľku. V tabuľke OSOBA je stĺpec ADRESA_ID (cudzí kľúč).

```
@Entity
```

```

@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
class Osoba {
    @Id
    long id;
    String meno;
    @JoinColumn(nullable = false, unique = true) // unique = 1:1
    Adresa adresa;
}

@Entity
class Adresa {
    @Id
    long id;
    String mesto;
    String ulica;
}

```

Riešenie 3 — Adresa ako entita, FK v Adrese

FK je opačne — v tabuľke ADRESA je stĺpec OSOBA_ID, ktorý ukazuje na OSOBA.

```

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
class Osoba {
    @Id long id;
    String meno;
}

@Entity
class Adresa {
    @Id long id;
    String mesto;
    String ulica;
    @JoinColumn(nullable = false, unique = true)
    Osoba osoba; // FK na OSOBA – opačný smer ako v riešení 2
}

```

Bodovanie Úlohy A — 8 bodov

Čo sa hodnotí	Body	Poznámka
Správne použitie extends (dedičnosť)	1 bod	FyzickaOsoba extends Osoba
Správne anotácie @Entity a @Inheritance	1 bod	Ak chýba jedna z týchto → 0 bodov
Správna M:N asociácia zamestnanec-zamestnávateľ	3 body	mappedBy + List na oboch stranách
Správna 1:1 asociácia Osoba-Adresa	3 body	@Embedded/@JoinColumn s unique

3. Úloha B — Schéma databázy (4 body)

ZADANIE

- Pre návrh z časti A) napíšte schému databázy.
- Aké tabuľky a stĺpce vzniknú? Aké sú medzi nimi väzby (FK, prepojovacie tabuľky)?

Schéma pre Riešenie 1 (Adresa @Embeddable)

Vzniknú 2 tabuľky: OSOBA (so všetkými stĺpcami vrátane mesta a ulice z Adresy) + prepojovacia tabuľka.

Tabuľka: OSOBA

Stĺpec	Typ	Popis
id	bigint (PK)	Primárny kľúč
DTYPE	varchar	Diskriminátor: 'FyzickaOsoba' alebo 'PravnickaOsoba'
meno	varchar	Spoločné pre obe podtriedy
mesto	varchar	Z @Embedded Adresa — stĺpec je priamo tu!
ulica	varchar	Z @Embedded Adresa — stĺpec je priamo tu!
rc	varchar	Len pre FyzickaOsoba (ostatní majú NULL)
narod	date	Len pre FyzickaOsoba (ostatní majú NULL)
ico	varchar	Len pre PravnickaOsoba (ostatní majú NULL)
dic	varchar	Len pre PravnickaOsoba (ostatní majú NULL)

Prepojovacia tabuľka: OSOBA_OSOBA (M:N vzťah)

Stĺpec	Typ	Popis
zamestnanec_id	bigint (FK)	Odkazuje na OSOBA.id
zamestnavatel_id	bigint (FK)	Odkazuje na OSOBA.id

 Oba FK ukazujú na TÚ ISTÚ tabuľku OSOBA — keďže FyzickaOsoba aj PravnickaOsoba sú tam.

Schéma pre Riešenie 2 (Adresa entita, FK v Osobe)

Vzniknú 3 tabuľky: OSOBA + ADRESA + prepojovacia tabuľka. FK adresa_id je v OSOBA.

Stĺpec	Typ	Popis
id	bigint (PK)	Primárny kľúč
DTYPE	varchar	Diskriminátor
meno	varchar	
adresa_id	bigint (FK, UNIQUE)	Cudzí kľúč → ADRESA.id, UNIQUE zaručuje 1:1
rc, narod, ico, dic	...	Ako v riešení 1

Schéma pre Riešenie 3 (FK v Adrese)

Rovnaké ako Riešenie 2 ale FK osoba_id je v tabuľke ADRESA (nie v OSOBA).

Tabuľka ADRESA — stĺpce	Typ	Popis
id	bigint (PK)	
mesto	varchar	
ulica	varchar	
osoba_id	bigint (FK, UNIQUE)	FK → OSOBA.id, UNIQUE zaručuje 1:1

Bodovanie Úlohy B — 4 body

Čo sa hodnotí	Body
Správna prepojovacia tabuľka pre M:N	2 body
DTYPE (diskriminátor) v tabuľke OSOBA	1 bod
Správna tabuľka OSOBA (rieš.1) resp. OSOBA + ADRESA (rieš. 2 a 3)	1 bod

4. Úlohy C1, C2, C3 — JPQL a SQL dotazy

4.1 Teória — JPQL vs SQL

Vlastnosť	JPQL	SQL
Pracuje s	Entitami (Java triedami)	Tabuľkami v DB
FROM klauzula	FROM FyzickaOsoba o	FROM osoba o
Atribúty	o.adresa.mesto (cez asociáciu)	Treba manuálny JOIN
Pozná dedičnosť?	✓ Automaticky	X Treba WHERE dtype='...'
Navigácia	o.adresa.mesto funguje priamo	Musíš spojiť tabuľky ručne

C1) JPQL dotaz — 1 bod

ZADANIE C1

- Napíšte JPQL dotaz, ktorý vráti všetky mestá, v ktorých má adresu nejaká fyzická osoba.

```
SELECT o.adresa.mesto FROM FyzickaOsoba o WHERE o.adresa != null

// Poznámka: Podstatné je o.adresa.mesto a FROM FyzickaOsoba
// WHERE o.adresa != null je ochrana pred NullPointerException
// JPQL automaticky filtruje len fyzické osoby – nepotrebuje dtype!
```

💡 JPQL vie navigovať cez asociácie cez bodku (o.adresa.mesto). SQL toto nevie — treba JOIN.

C2) SQL dotaz pre SINGLE_TABLE — 1 bod

ZADANIE C2

- Napíšte SQL dotaz, ktorý vráti všetky mestá fyzických osôb.
- Predpokladajte stratégiu SINGLE_TABLE (jedna tabuľka OSOBA s DTYPE).

Riešenie 1 — Adresa @Embeddable (mesto priamo v tabuľke osoba)

```
SELECT o.mesto FROM osoba o WHERE o.dtype = 'FyzickaOsoba'

// Mesto je priamo v tabuľke osoba (z @Embedded)
// Žiadny JOIN nie je potrebný
```

Riešenie 2 — Adresa ako entita, FK adresa_id v OSOBA

```
SELECT a.mesto FROM adresa a, osoba o
WHERE o.dtype = 'FyzickaOsoba'
AND o.adresa_id = a.id

// Treba JOIN s tabuľkou adresa
```

```
// o.adresa_id je FK v tabuľke osoba
```

Riešenie 3 — Adresa ako entita, FK osoba_id v ADRESA

```
SELECT a.mesto FROM adresa a, osoba o
WHERE o.dtype = 'FyzickaOsoba'
AND    a.osoba_id = o.id

// FK je tentokrát v tabuľke adresa!
```

C2) SQL dotaz pre JOINED — variant z druhej verzie skúšky

Ak bola použitá stratégia JOINED, fyzická osoba má vlastnú tabuľku fyzickaosoba:

Riešenie 1 — Adresa @Embeddable (mesto v tabuľke fyzickaosoba)

```
SELECT f.mesto FROM fyzickaosoba f

// Pri JOINED má FyzickaOsoba vlastnú tabuľku fyzickaosoba
// Žiadny dtype filter – tabuľka obsahuje len fyzické osoby
```

Riešenie 2 — Adresa ako entita, FK v fyzickaosoba

```
SELECT a.mesto FROM adresa a, fyzickaosoba o
AND    o.adresa_id = a.id
```

Riešenie 3 — FK osoba_id v adresa

```
SELECT a.mesto FROM adresa a, fyzickaosoba o
AND    a.osoba_id = o.id
```

💡 Podstatné je: V riešeniach 2 a 3 join tabuliek Adresa a FyzickaOsoba. V riešení 1 žiadny join — selektuje sa len tabuľka FyzickaOsoba.

C3) SQL dotaz pre JOINED stratégiu — 1 bod

ZADANIE C3

- Napíšte SQL dotaz pre mestá fyzických osôb, ak bola použitá stratégia JOINED.

```
SELECT a.mesto FROM adresa a, osoba o
WHERE o.dtype = 'FyzickaOsoba'
AND    o.osoba_id = a.id

// Poznámka: môže byť aj trochu inak ale podstatné je:
// o.dtype = 'FyzickaOsoba'
```

💡 V rôznych verziách riešenia to môže byť aj trochu inak — podstatné je filtrovanie podľa `dtype='FyzickaOsoba'`.

5. Úloha 2 — JAXB serializácia do XML (6 bodov)

5.1 Teória — JAXB

JAXB (Java Architecture for XML Binding) je štandard pre konverziu Java objektov na XML (marshalling) a späť (unmarshalling).

Anotácia	Význam
@XmlRootElement	Trieda môže byť koreňový element v XML
@XmlElement	Atribút sa serializuje ako vnorený XML element (default)
@XmlAttribute	Atribút sa serializuje ako XML atribút <tag attr="..."/>
@XmlTransient	Atribút sa NESERIALIZUJE — preskočí sa úplne
@XmlElementWrapper	Vytvorí obalovací element okolo kolekcie

ZADANIE

- Navrhните dátové triedy Osoba a Poistenie pre serializáciu do XML cez JAXB.
- Najdôležitejšie: triedy musia umožňovať PRIRADIŤ poistené osoby poistným zmluvám.
- Trieda Poistenie musí obsahovať referenciu na majiteľa poistenia.

Riešenie 1 — Zoznam poistencov vnútri triedy Poistenie (@XmlTransient)

Trieda Poistenie obsahuje zoznam poistencov, ale označený @XmlTransient — nezapíše sa do XML.

```
@XmlRootElement
class Osoba {
    String meno;
    String rc;
    String adresa;
}

@XmlRootElement
class Poistenie {
    String idZmluvy;
    Osoba majitel;           // zapíše sa do XML ako vnorený element
    int pocetPoistencov;
    @XmlTransient           // NEZAPÍŠE sa do XML — preskočí sa!
    List<Osoba> poistenci;
}

// Mimo tried — kontajner pre všetky poistenia:
List<Poistenie> zoznamPoisteni;
// alebo
Map<String, Poistenie> zoznamPoisteni; // kľúč = idZmluvy
```

Prečo @XmlTransient na zozname poistencov?

Bez @XmlTransient by JAXB pokúšal serializovať každú osobu znova ako celý objekt. Problémy:

- Duplicita — tá istá osoba sa zapíše viackrát (v každom poistení kde je)
- Neefektívne — XML by bol obrovský
- Možné cykly — ak by Osoba odkazovala späť na Poistenie

Výsledné XML pre Riešenie 1:

```
<poistenie>
  <idZmluvy>P001</idZmluvy>
  <majitel>
    <meno>Jano Novák</meno>
    <rc>800101/1234</rc>
    <adresa>Hlavná 5, Bratislava</adresa>
  </majitel>
  <pocetPoistencov>3</pocetPoistencov>
  <!-- poistenci sa NEZAPÍŠU — sú @XmlTransient -->
</poistenie>
```

Riešenie 2 — Priradenie poistencov cez externé Map a List

Trieda Poistenie neobsahuje zoznam poistencov vôbec. Priradenie sa rieši externými dátovými štruktúrami.

```
@XmlRootElement
class Osoba {
    String meno;
    String rc;
    String adresa;
}

@XmlRootElement
class Poistenie {
    String idZmluvy;
    Osoba majitel;
    int pocetPoistencov;
    // Žiadny zoznam poistencov tu!
}

// Externé štruktúry pre priradenie:
Map<String, Poistenie> zoznamPoisteni;           // kľúč = idZmluvy
Map<String, List<Osoba>> poistenci;              // kľúč = idZmluvy → zoznam
poistených
// alebo
Map<Poistenie, List<Osoba>> mapaPoisteni;       // priame mapovanie
```

Bodovanie Úlohy 2 — 6 bodov

Riešenie	Body	Podmienka
Úplne správne	6 bodov	Všetko správne — majitel, @XmlRootElement, priradenie funguje

Riešenie	Body	Podmienka
Takmer správne	4–5 bodov	1–2 drobné nedostatky (chýba @XmlTransient, chýba deklarácia zoznamu poistení...)
Funguje priradenie, ale inak zlé	3 body	Priradenie poistencov je možné ale riešenie má väčšie nedostatky
Neumožňuje priradenie	max 2 body	1 bod za majitel, 1 bod za @XmlRootElement na oboch triedach

6. REST — Prekladový slovník

6.1 Teória — REST základy

REST (Representational State Transfer) je architektonický štýl pre webové API. Princípy:

- — **všetko je zdroj (URI)**
- — **server si nepamätá stav klienta medzi požiadavkami**
- — **GET, POST, PUT, DELETE** pre rôzne operácie

Metóda	Účel	Idempotentná?	Príklad
GET	Čítanie zdroja — nič nemení	✓ Áno	GET /dictionary/sk/dog
POST	Vytvorenie — server určí URI	✗ Nie	POST /users
PUT	Vytvorenie alebo úprava na URI	✓ Áno	PUT /dictionary/sk/dog
DELETE	Odstránenie zdroja	✓ Áno	DELETE /dictionary/sk
PATCH	Čiastočná úprava	✗ Nie	PATCH /user/1

💡 *Idempotentnosť* = volanie viackrát dáva rovnaký výsledok ako jedno volanie. PUT je idempotentný — preto sa používa namiesto POST keď klient pozná URI.

6.2 URI štruktúra

```
/dictionary           // koreňový resource — všetky slovníky
/dictionary/{lang}    // konkrétny jazyk (napr. /dictionary/sk)
/dictionary/{lang}/{word} // konkrétne slovo (napr. /dictionary/sk/dog)
```

Pravidlá URI

- ✓ Podstatné mená: /dictionary (nie /getDictionary)
- ✓ Hierarchia vyjadruje vzťah: /dictionary/{lang}/{word}
- ✓ Plurál pre kolekcie: /users, /products
- ✗ Nie slovesá v URI — sloveso vyjadruje HTTP metóda!

📋 ZADANIE

- Implementovať REST servis — prekladový slovník z angličtiny do viacerých jazykov.
- GET /dictionary → zoznam podporovaných jazykov
- GET /dictionary/{lang}/{word} → vráti preklad slova
- PUT /dictionary/{lang}/{word} → pridá/upraví preklad
- DELETE /dictionary/{lang} → zmaže celý slovník pre jazyk
- DELETE /dictionary/{lang}/{word} → zmaže preklad slova
- a) Navrhnete dátovú štruktúru v pamäti (Java)
- b) Navrhnete štruktúru relačnej databázy

6.3 Riešenie a) — Dátová štruktúra v pamäti

Najjednoduchšie a najefektívnejšie riešenie:

```
Map<String, Map<String, String>> dictionary;
//   ^lang           ^eng_word ^translation

// Vonkajšia mapa: kľúč = jazyk (napr. "sk")
// Vnútoraná mapa: kľúč = anglické slovo (napr. "dog"), hodnota = preklad ("pes")

// Použitie pre jednotlivé REST operácie:

// GET /dictionary - zoznam jazykov:
Set<String> jazyky = dictionary.keySet();

// GET /dictionary/{lang}/{word} - preklad:
String preklad = dictionary.get(lang).get(word);

// PUT /dictionary/{lang}/{word} - pridaj/uprav:
dictionary.computeIfAbsent(lang, k -> new HashMap<>()).put(word, translation);

// DELETE /dictionary/{lang} - zmaž slovník:
dictionary.remove(lang);

// DELETE /dictionary/{lang}/{word} - zmaž slovo:
dictionary.get(lang).remove(word);
```

Alternatívne riešenie (trieda Lang):

```
List<Lang> languages;

class Lang {
    String name;           // napr. "sk"
    Map<String, String> words; // anglické slovo → preklad
}

// Funkčne správne, ale vyhľadanie jazyka je O(n) - prehľadáva celý zoznam.
// Map<String, Lang> by bolo lepšie - O(1) vyhľadanie podľa názvu jazyka.
```

💡 *Map<String, Map<String, String>> je efektívnejšie ako List<Lang> pre vyhľadávanie O(1) vs O(n).*

6.4 Riešenie b) — Schéma relačnej databázy

Normalizovaný dizajn — každá entita v samostatnej tabuľke, preklady v asociačnej tabuľke.

Tabuľka: language

Stĺpec	Typ	Popis
id	UUID / bigint (PK)	Primárny kľúč
name	VARCHAR	Názov jazyka napr. 'sk', 'de', 'fr'

Tabuľka: eng_word

Stĺpec	Typ	Popis
id	UUID / bigint (PK)	Primárny kľúč
word	VARCHAR	Anglické slovo napr. 'dog', 'cat'

Tabuľka: dictionary (preklady)

Stĺpec	Typ	Popis
word_id	bigint (FK, PK)	Odkazuje na eng_word.id
lang_id	bigint (FK, PK)	Odkazuje na language.id
translation	VARCHAR	Preklad napr. 'pes'

💡 Primárny kľúč tabuľky dictionary je ZLOŽENÝ: (word_id, lang_id). Zaručuje, že pre danú dvojicu (slovo, jazyk) existuje práve jeden preklad.

6.5 REST implementácia v Java — JAX-RS

```
@Path("/dictionary")
public class DictionaryService {

    private Map<String, Map<String, String>> dictionary = new HashMap<>();

    // GET /dictionary - zoznam jazykov
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getLanguages() {
        return String.join(",", dictionary.keySet());
    }

    // GET /dictionary/{lang}/{word} - preklad
    @GET
    @Path("/{lang}/{word}")
    public String getTranslation(@PathParam("lang") String lang,
                                @PathParam("word") String word) {
        return dictionary.get(lang).get(word);
    }

    // PUT /dictionary/{lang}/{word} - pridaj alebo uprav
    @PUT
    @Path("/{lang}/{word}")
    @Consumes(MediaType.TEXT_PLAIN)
    public Response setTranslation(@PathParam("lang") String lang,
                                    @PathParam("word") String word,
                                    String translation) {
        dictionary.computeIfAbsent(lang, k -> new HashMap<>()).put(word,
translation);
        return Response.ok().build();
    }

    // DELETE /dictionary/{lang} - zmaž celý slovník
    @DELETE
    @Path("/{lang}")
    public Response deleteLanguage(@PathParam("lang") String lang) {
        dictionary.remove(lang);
    }
}
```

```

        return Response.ok().build();
    }

    // DELETE /dictionary/{lang}/{word} - zmaž jedno slovo
    @DELETE
    @Path("/{lang}/{word}")
    public Response deleteWord(@PathParam("lang") String lang,
                               @PathParam("word") String word) {
        if (dictionary.containsKey(lang)) dictionary.get(lang).remove(word);
        return Response.ok().build();
    }
}

```

JAX-RS anotácie — prehľad

Anotácia	Význam
@Path("...")	Definuje URI cestu
@GET, @PUT, @POST, @DELETE	HTTP metóda
@PathParam("...")	Parameter z URI cesty (napr. {lang})
@QueryParam("...")	Parameter z query stringu ?key=value
@Produces(MediaType.TEXT_PLAIN)	Aký formát vracia (JSON, XML, plain text)
@Consumes(MediaType.APPLICATION_JSON)	Aký formát prijíma

7. Ďalšie varianty skúšky — Predmet, Profesor, Ucitel

Na skúške sa môžu objaviť rôzne varianty s rôznymi triedami ale rovnakou logikou. Typický príklad:

ZADANIE — variant so stratégiou SINGLE_TABLE

- a) Navrhните a nakreslite štruktúru databázy pre entitné triedy znázornené UML diagramom.
- Pre mapovanie hierarchie tried zvolte stratégiu SINGLE_TABLE.
- b) Implementujte entitné triedy podľa UML diagramu a anotujte ich.
- UML: Predmet (id, nazov) ↔ M:N ↔ Ucitel/Osoba (id, meno) ← dedí Profesor/Docent (ustav)
- Vzťah garant: Predmet M:1 Profesor (jeden predmet má jedného garanta)
- Vzťah cviciaci: Predmet M:N Ucitel (predmet môže mať viac cvičiacich)

Riešenie — SINGLE_TABLE

a) Schéma databázy:

Tabuľka	Stĺpce	Poznámka
OSOBA / UCITEL	id (PK), DTYPE, meno, ustav	ustav je NULL pre Ucitel, vyplnené pre Docent/Profesor
PREDMET	id (PK), nazov, garant_id (FK)	garant_id → OSOBA.id
PREDMET_OSOBA	predmet_id (FK), osoba_id (FK)	Prepojovacia tabuľka pre M:N cviciaci

b) Java kód — SINGLE_TABLE:

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
class Osoba {
    @Id Long id;
    String meno;
}

@Entity
class Profesor extends Osoba {
    String ustav;
}

@Entity
class Predmet {
    @Id Long id;
    String nazov;

    @ManyToMany
    List<Osoba> cviciaci;

    @ManyToOne
    Profesor garant;
}
```

Riešenie — JOINED

a) Schéma databázy pre JOINED:

Tabuľka	Stĺpce	Poznámka
OSOBA	id (PK), DTYPE, meno	Spoločné atribúty nadtriedy
PROFESOR	id (PK+FK → OSOBA), ustav	id je FK na OSOBA.id
PREDMET	id (PK), nazov, garant_id (FK)	garant_id → OSOBA.id
PREDMET_OSOBA	predmet_id (FK), osoba_id (FK)	Prepojovacia tabuľka M:N

b) Java kód — JOINED:

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
class Osoba {
    @Id Long id;
    String meno;
}

@Entity // Vznikne samostatná tabuľka PROFESOR
class Profesor extends Osoba {
    String ustav;
}

@Entity
class Predmet {
    @Id Long id;
    String nazov;

    @ManyToMany
    List<Osoba> cviciaci;

    @ManyToOne
    Profesor garant;
}
```

Riešenie — TABLE_PER_CLASS

a) Schéma databázy pre TABLE_PER_CLASS:

Tabuľka	Stĺpce	Poznámka
OSOBA	id (PK), meno	Len ak Osoba nie je abstraktná
PROFESOR	id (PK), meno, ustav	Obsahuje aj zdedené stĺpce z Osoba!
PREDMET	id (PK), nazov, garant_id (FK)	
PREDMET_OSOBA	predmet_id (FK), osoba_id (FK)	Prepojovacia tabuľka M:N

💡 Pri TABLE_PER_CLASS každá konkrétna trieda má VŠETKY stĺpce vrátane zdedených. Žiadne NULL hodnoty, ale pri polymorfných dotazoch treba UNION.

b) Java kód — TABLE_PER_CLASS:

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
class Osoba {
    @Id Long id;
    String meno;
}

@Entity // Tabuľka PROFESOR má stĺpce: id, meno, ustav
class Profesor extends Osoba {
    String ustav;
}

@Entity
class Predmet {
    @Id Long id;
    String nazov;
    @ManyToMany List<Osoba> cviciaci;
    @ManyToOne Profesor garant;
}
```

8. Rýchly prehľad — Ťahák na skúšku

8.1 JPA — Čo VŽDY treba

JPA — Povinné prvky

- @Entity na každej triede ktorá má tabuľku
- @Id na primárnom kľúči (každá entita musí mať)
- @Inheritance(strategy = ...) na nadtriede hierarchie
- extends v Jave vyjadruje dedičnosť tried
- @ManyToMany + mappedBy na jednej strane pre M:N
- @JoinColumn(nullable=false, unique=true) pre 1:1 cez FK

JPA — Časté chyby

- Zabudnúť mappedBy → vzniknú 2 prepojovacie tabuľky!
- Zabudnúť @Entity alebo @Inheritance → 0 bodov za celú triedu
- Dať mappedBy na obe strany → JPA nebude vedieť kto je vlastník
- Zamieniť @Embeddable a @Embedded → @Embeddable je na triede, @Embedded na atribúte

8.2 SQL — Podľa stratégie

Stratégia	Tabuľka pre fyz. osoby	Filter	JOIN s adresou?
SINGLE_TABLE + @Embedded	osoba	WHERE o.dtype='FyzickaOsoba'	Nie — mesto je v osoba
SINGLE_TABLE + @Entity Adresa (FK v osoba)	osoba	WHERE o.dtype='FyzickaOsoba'	Áno — JOIN cez o.adresa_id=a.id
SINGLE_TABLE + @Entity Adresa (FK v adresa)	osoba	WHERE o.dtype='FyzickaOsoba'	Áno — JOIN cez a.osoba_id=o.id
JOINED + @Embedded	fyzickaosoba	Žiadny — vlastná tabuľka	Nie — mesto je v fyzickaosoba
JOINED + @Entity Adresa	fyzickaosoba	Žiadny — vlastná tabuľka	Áno — JOIN cez o.adresa_id=a.id

8.3 JAXB — Čo VŽDY treba

JAXB — Povinné prvky

- @XmlRootElement na oboch triedach (Osoba aj Poistenie)
- Osoba majitel v triede Poistenie (referencia na majiteľa)
- Spôsob priradenia poistencov k poisteniu (List alebo Map)
- @XmlTransient ak je zoznam poistencov vnútri triedy

8.4 REST — Top 5 pravidiel

- — /dictionary nie /getDictionary

- — použiť keď klient pozná presné URI kde uložiť
- — len číta, nikdy nesmie meniť stav
- — `@PathParam("lang") String lang`
- — môže sa volať viackrát bezpečne (idempotentné)

8.5 Porovnanie JPQL vs SQL

Aspekt	JPQL	SQL
Syntax FROM	FROM FyzickaOsoba o	FROM osoba o WHERE dtype='...'
Navigácia	o.adresa.mesto (priamo)	Treba JOIN s tabuľkou adresa
Dedičnosť	Automatická — FROM FyzickaOsoba filtruje	Manuálny WHERE dtype filter
JOIN-y	Automatické cez asociácie	Manuálne — treba písať ručne

Veľa šťastia na skúške! 